

תכנות מתקדם (Python)

נקודות זכות: 5

סוג הקורס: קורס חובה

דרישות קדם: מבוא לתכנות ותכנות מונחה עצמים בשפת Python

שם המרצה: ד"ר סופיה אמדור נלקה

מטרות הקורס

הקורס מחבר בין הידע שנרכש במבוא לתכנות ובתכנות מונחה עצמים לבין עבודה מקצועית בפרויקט תוכנה. במהלך הקורס הסטודנטים יתרגלו עבודה עם Git ו-GitHub, צריכה אמינה של ממשקי API, ארגון קוד ותצורה, תבניות תכנון שימושיות, בדיקות אוטומטיות ותכנות מקבילי ואסינכרוני. הדגש הוא על בניית קוד מודולרי, ניתן לבדיקה ולתחזוקה, ועל הכנה להמשך לימודים בפיתוח Frontend ו-Backend.

תפוקות למידה

בסיום הקורס בהצלחה הסטודנט:

- ידע לנהל קוד וגרסאות באמצעות Git ו-GitHub, לרבות ענפים, מיזוגים ו-Pull Requests.
- ידע לארגן פרויקט Python באמצעות סביבה וירטואלית, מודולים, חבילות, תצורה ומשתני סביבה.
- ידע לצרוך ממשקי REST API, לעבד JSON ולטפל באימות, שגיאות, timeouts ו-pagination.
- ידע ליישם עקרונות OOP ותבניות תכנון נבחרות לצורך קוד גמיש וניתן לתחזוקה.
- ידע לכתוב בדיקות אוטומטיות ולדמות תלויות חיצוניות, לרבות קריאות API.
- ידע לבחור וליישם threads, processes או async/await בהתאם למשימה ולנתח סיכוני מקביליות.

שיטות הלימוד

הקורס משלב הסבר קצר, הדגמה ותרגול מעשי בכל מפגש. לאורך הסמסטר הסטודנטים יפתחו פרויקט מצטבר ב-GitHub: ינהלו גרסאות וענפים, יתחברו ל-API חיצוני, יכתבו בדיקות ובהמשך ישפרו את הביצועים באמצעות כלים של תכנות מקבילי ואסינכרוני. החזרה על חומרי הקדם תתבצע דרך משימות אבחון ורפקטורינג ולא כשחזור מלא של הקורסים הקודמים.

הערכת הסטודנט והרכב הציון

70%

מבחן מסכם

30%

תרגילים שוטפים ופרויקט אינטגרטיבי ב-GitHub

תנאי למעבר הקורס הוא ציון עובר (60) בבחינה.

נושאי הקורס

נושאים	שיעור
חזרה ממוקדת על Python ו-OOP והשלמת חומרי הקדם: מחלקות ואובייקטים, הורשה והרכבה, overriding, polymorphism, encapsulation, property, dunder methods, class/static, דקורטורים ומתודות.	1
Git, GitHub וארגון פרויקט: Pull repository, add/commit, remote, clone/push/pull, branches, merge, Requests; עבודה עם venv, מודולים, חבילות, README, תיעוד ו-gitignore.	2

שיעור	נושאים
3	מבני נתונים ותכנות פונקציונלי: שימושים מתקדמים ב-set ו-dict, מבני heap ו-deque, סיבוכיות בסיסית; comprehensions ו-unpacking מתקדם, map, filter, lambda ו-reduce.
4	כלי Python מתקדמים: iterators, generators ו-lazy evaluation; העמקה בדקורטורים ו-context managers ויישומם בקוד מודולרי.
5	עבודה עם API וטיפול מקצועי בשגיאות: HTTP ו-JSON, headers, status codes, methods, REST ו-query authentication; API keys ו-parameters, קובצי תצורה ומשתני סביבה, validation, timeouts, retries, חריגות ולוגים.
6	Design Patterns ותכנון תוכנה: Singleton, Factory, Observer ו-Strategy; בחירת תבנית לפי הבעיה, composition ו-dependency injection, ודיון גם במגבלות של Singleton.
7	בדיקות אוטומטיות: unit tests באמצעות parametrization, fixtures, pytest, בדיקת חריגות, mocking של תלויות ושל קריאות API ועקרונות TDD.
8	פרופילינג ואופטימיזציה: מדידת זמן, זיכרון וזיהוי צווארי בקבוק; שימוש בכלי profiling, בחירת מבנה נתונים מתאים והבחנה בין שיפור מדיד לבין אופטימיזציה מוקדמת.
9	תכנות מקבילי באמצעות threads ו-processes: משימות I/O לעומת CPU, race conditions, GIL ו-locks; עבודה עם ThreadPoolExecutor, ProcessPoolExecutor ו-multiprocessing.
10	תכנות אסינכרוני ואינטגרציה: asyncio, async/await, tasks, gather ו-timeouts; ביצוע קריאות API במקביל, מדידת השיפור ושילוב Git, בדיקות וארגון קוד בפרויקט המסכם.
11	חזרה על חומר הקורס והכנה למבחן: פתרון תרגיל אינטגרטיבי, חזרה על מושגי מפתח, השוואה בין חלופות תכנון ומודלים של מקביליות ומענה על שאלות.

ייתכנו שינויים בתכנית הלימודים.

חומרי קריאה ומשאבים

- Pro Git / Scott Chacon & Ben Straub
- Python Documentation — venv, pathlib, asyncio, concurrent.futures ו-multiprocessing
- GitHub Skills
- pytest Documentation
- HTTPX Documentation
- Design Patterns in Python